

OpenStack

Système de Gestion de Cloud (CMS)

UE CRV — Sorbonne Université / LIP6

`Naceur.Malouch@lip6.fr`

Cours 5 : Composants, services et exemples d'usage

Table des matières

1	Introduction à OpenStack	2
2	Composants OpenStack	2
2.1	Services principaux	2
2.2	Outils d'exploitation (admin)	2
2.3	Outils d'intégration	3
3	Architecture Physique	3
3.1	Types de nœuds	3
3.2	Types de réseaux physiques	3
3.3	Configuration matérielle minimale (production)	3
3.4	Variantes d'architecture	3
4	NEUTRON – Service de Networking	4
4.1	Composants Neutron	4
4.2	Neutron avec OVN (Open Virtual Network)	4
4.2.1	Composants par nœud	4
4.2.2	Configuration ML2 avec OVN	4
4.3	Types de réseaux Neutron	5
4.4	Exemples de commandes Neutron	5
5	HORIZON – Dashboard	5
6	NOVA – Compute Service	5
6.1	Sous-composants de Nova	6
6.2	Hyperviseurs supportés	6
6.3	Images – Glance	6
6.4	Gabarits (Flavors)	6
6.5	Projets, tenants et RBAC	7
7	KEYSTONE – Identity Service	7
8	CINDER – Block Storage Service	7
9	SWIFT – Object Store	8
10	HEAT – Orchestration Service (IaC)	8
10.1	Exemple de template HOT	8
11	Environnement TME – DevStack	9
12	Sources	10

Introduction à OpenStack

OpenStack – Définition

OpenStack est un ensemble de composants logiciels **open source** qui fournissent des services communs pour une infrastructure cloud (public ou privé).

Caractéristiques principales :

- Peut être déployé sur plusieurs architectures physiques de centres de données connectés
- Principalement connu pour le modèle **IaaS**, mais supporte aussi d'autres services : orchestration, gestion des pannes, etc.
- Facilite le contrôle de grands groupements de ressources :
 - **Compute** : ressources de calcul (VMs, bare metal, conteneurs)
 - **Storage** : stockage bloc, objet, fichier
 - **Networking** : réseaux virtuels, routeurs, load balancers
- Passage à l'échelle vertical (scale up / scale down) et horizontal (scale out / scale in)

Composants OpenStack

Services principaux

Nom	Service	Rôle
NOVA	Compute Service	Gestion des instances (VMs, bare metal, conteneurs)
IRONIC	Bare Metal provisioning	Provisionnement de serveurs physiques
NEUTRON	Networking service	Création et gestion des réseaux virtuels
CINDER	Block Storage service	Volumes de stockage persistants
SWIFT	Object Storage service	Stockage distribué d'objets
KEYSTONE	Identity service	Authentification, autorisation, catalogue de services
HORIZON	Dashboard (Interface Web)	Interface graphique Web pour tous les services
GLANCE	Image service	Dépôt des images disque des instances
BLAZAR	Resource reservation	Réservation de ressources
HEAT	Infrastructure-as-Code	Déploiement de ressources via templates

Outils d'exploitation (admin)

- **CEILOMETER** : Metering & Data Collection
- **WATCHER** : Optimization Service
- **CLOUDKITTY** : Billing and chargebacks
- **TEMPEST** : The OpenStack Integration Test Suite
- **RALLY** : Benchmarking tool

Outils d'intégration

- **KURYR** : OpenStack Networking integration for containers
- **TACKER** : NFV Orchestration

Voir tous les projets : <https://www.openstack.org/software/project-navigator/>

Architecture Physique

Types de nœuds

Nœud	Rôle
Controller node (1–3)	Exécute Identity service, Image service, management Compute & Networking, dashboard, base de données SQL
Network node (1–3)	Exécute les agents DHCP et Metadata, plug-in de création de réseaux virtuels, autres utilitaires réseau
Compute node (1–n)	Exécute l'hyperviseur qui opère les instances ; exécute un agent pour le service networking
Block Storage node	Contient les disques utilisés par les services Block Storage et Shared File System

Types de réseaux physiques

- **Management Network** : Communication entre les composants de gestion (ex : Nova controller ↔ Nova agent)
- **External Network** : Connecte les VMs à Internet via les routeurs virtuels
- **Data Network** : Connecte les VMs entre elles et vers les ressources réseaux (routeurs virtuels, etc.)

Les nœuds de contrôle sont généralement connectés uniquement au réseau de gestion (+ éventuellement une IP publique sur le réseau externe). Les autres nœuds peuvent être connectés à tous les réseaux.

Configuration matérielle minimale (production)

Node	CPU	RAM	Stockage	Interfaces
Controller	1–2	8 GB	100+ GB	Management
Compute	2–4+	8+ GB	100+ GB	Management + Overlay (+ Internet opt.)
Database	1–2	8 GB	100+ GB	Management
Gateway	1–2	8 GB	100+ GB	Management + Overlay + Internet

Variantes d'architecture

Il est possible de décomposer le Network Node en :

- **Database Node** : bases de données des réseaux logiques
- **Gateway Node** : opérations réseaux explicites (NAT, routage externe)

Ou encore de combiner Database Node + Controller Node → retour à 3 nœuds : Controller, Gateway, Compute.

NEUTRON – Service de Networking

Neutron est un projet selon l'approche **SDN** fournissant les services réseaux (*NaaS* – *Network as a Service*).

Fonctionnalités :

- Crée et connecte les interfaces de périphériques aux réseaux
- Intègre différents équipements et logiciels réseau
- Augmente la flexibilité de l'architecture et du déploiement d'OpenStack
- Interagit principalement avec Nova pour fournir la connectivité aux instances

Composants Neutron

- **neutron-server** : reçoit les requêtes API et les envoie au plug-in réseau approprié
- **Networking plug-ins and agents** : créent les réseaux virtuels et les configurations des sous-réseaux (dépend du fournisseur : NEC OpenFlow, Open vSwitch + bridges Linux, OVN...)
- **Messaging queue** : communication entre les composants de Neutron ; peut stocker l'état du réseau pour certains plug-ins

Neutron avec OVN (Open Virtual Network)

Composants par nœud

Nœud	Composants
Controller	neutron-server, ML2 Plug-in, OVN Mechanism Driver, OVN Layer-3 Service Plug-in
Database	OVN Northbound Service (ovn-northd), OVN Northbound DB (ovnnb.db), OVN Southbound DB (ovnsb.db)
Gateway	OVN Controller (ovn-controller), OVS Local DB (ovsdb-server), OVS Data Plane (ovs-vswitchd)
Compute	OVN Controller, OVS Local DB, OVN Metadata Agent, OVS Data Plane

Configuration ML2 avec OVN

```
# /etc/neutron/plugins/ml2/ml2_conf.ini
[ml2]
type_drivers = local,flat,vlan,geneve
mechanism_drivers = ovn
tenant_network_types = geneve
extension_drivers = port_security,qos
```

Types de réseaux Neutron

External	Shared	Type de réseau
✓	×	Provider Network : connecte les VMs à Internet via des routeurs virtuels
✓	✓	Shared Provider Network
×	×	Tenant Network : connecte les VMs entre elles (VPC – Virtual Private Cloud)
×	✓	Shared Tenant Network

Exemples de commandes Neutron

Créer un réseau :

```
stack$ openstack network create selfservice
```

Lister les réseaux :

```
stack$ openstack network list
```

Afficher les détails d'un réseau :

```
stack$ openstack network show public
# provider:network_type = flat (reseau externe)
# provider:network_type = geneve (reseau interne/overlay)
```

HORIZON – Dashboard

- Implémentation du tableau de bord Web d'OpenStack
- **Extensible** : on peut ajouter de nouveaux composants
- Fournit une interface utilisateur Web aux services : Neutron, Nova, Swift, Keystone, etc.
- Basé sur **Python** et **Django**

Django : environnement de développement Web Python de haut niveau facilitant le développement d'applications web WSGI.

NOVA – Compute Service

- Fournit l'accès aux ressources de calcul : bare metal, VMs, conteneurs
- Conçu pour être évolutif, à la demande et en libre-service
- Dépendances avec d'autres services :**
 - **Keystone** : identité et authentification
 - **Glance** : dépôt d'images des instances
 - **Neutron** : provisionnement des réseaux auxquels se connectent les instances
 - **Placement** : suivi des ressources disponibles

Sous-composants de Nova

Composant	Rôle
DB	Base de données SQL pour stocker les données Nova
API	Reçoit les requêtes HTTP, convertit les commandes, communique via <code>oslo.messaging</code>
Scheduler	Décide quel hôte exécute chaque instance
Compute	Gère les communications entre l'hyperviseur et les machines virtuelles
Conductor	Coordonne les demandes complexes (build/resize), proxy de base de données

Hyperviseurs supportés

- Baremetal (IRONIC)
- **KVM** (Kernel-based Virtual Machine) – principal en développement
- LXC (Linux Containers)
- QEMU (Quick Emulator)
- VMware vSphere

Images – Glance

Les images des instances sont stockées par **Glance** (Image Service) :

```
$ openstack image list
```

ID	Name
Status	
ae1d242-730f-431f-88c1-87630c0f07ba	Ubuntu 14.04 cloudimg amd64
active	
0b27baa1-0ca6-49a7-b3f4-48388e440245	Ubuntu 14.10 cloudimg amd64
active	
df8d56fc-9cea-4dfd-a8d3-28764de3cb08	jenkins
active	

- **ID** : UUID de l'image (auto-généré)
- **Name** : nom quelconque
- **Status** : `active` → image utilisable
- **Server** : UUID de l'instance source si c'est un snapshot

Gabarits (Flavors)

Les modèles de systèmes/matériels virtuels sont appelés **gabarits** (*flavors*) :

```
$ openstack flavor list
```

ID	Name	RAM	Disk	Ephemeral	VCPUs	Is_Public
----	------	-----	------	-----------	-------	-----------

1	m1.tiny	512	1	0	1	True
2	m1.small	2048	20	0	1	True
3	m1.medium	4096	40	0	2	True
4	m1.large	8192	80	0	4	True
5	m1.xlarge	16384	160	0	8	True

Projets, tenants et RBAC

- Nova utilise la notion de **projet** (*tenant* = locataire)
- Accès géré par **RBAC** (Role-Based Access Control)
- Des **quotas** peuvent limiter le nombre de vCPUs et la RAM par tenant

KEYSTONE – Identity Service

Fonctionnalités :

- Validation des identifiants d'authentification ; création et renvoi de **tokens** en cas de succès
- Création et gestion des **utilisateurs, groupes, projets** et **domaines**
- Les **rôles** peuvent être attribués aux utilisateurs ou groupes, au niveau du domaine ou du projet
- Fournit le **catalogue de services** OpenStack (endpoints des APIs)

Technologies utilisées :

- Expose des méthodes HTTP : GET, PUT, POST, etc.
- Utilise la librairie **Flask-RESTful** pour l'interface REST API
- **Flask** : environnement Python d'applications Web WSGI (*Web Server Gateway Interface*)

Ressources :

- <https://flask-restful.readthedocs.io/>
- <https://flask.palletsprojects.com/>
- <https://wsgi.readthedocs.io/>

CINDER – Block Storage Service

- Gestion des périphériques de stockage
- Fournit une API pour demander/consommer des ressources de stockage
- Aucune connaissance requise de l'emplacement ou du type de stockage
- Base de données SQL centrale partagée par tous les services Cinder

Commandes principales :

```
openstack volume create
openstack volume delete
openstack volume list
openstack volume migrate
```


SWIFT – Object Store

- Stockage/archivage de données hautement disponible, distribué, avec consistance
- Permet de stocker de nombreuses données de manière efficace, sûre et “à moindre coût”
- Structure : **Objet** $\subset$ **Conteneur** $\subset$ **Compte**

Exemple :

```
openstack container create store1
openstack object create --name myres store1 /home/work/resImage.jpg
```

HEAT – Orchestration Service (IaC)

HEAT – Infrastructure-as-Code

Création et gestion des ressources d'infrastructure basées sur des **fichiers de templates** (HOT – Heat Orchestration Template).

- Un template décrit les applications cloud (actions à effectuer)
- Les descriptions permettent d'exécuter les appels d'API OpenStack appropriés
- Les modèles précisent les **relations entre les ressources** (ex : ce volume est connecté à ce serveur)
- S'intègre bien avec des outils de gestion de configuration comme **Ansible**

Exemple de template HOT

```
heat_template_version: 2013-05-23

description: >
  Hello world HOT template that just defines a single server.

parameters:
  key_name:
    type: string
    description: Name of an existing key pair to use for the server
    constraints:
      - custom_constraint: nova.keypair
  flavor:
    type: string
    description: Flavor for the server to be created
    default: m1.small
    constraints:
      - custom_constraint: nova.flavor
  image:
    type: string
    description: Image ID or image name to use for the server
    constraints:
      - custom_constraint: glance.image
  admin_pass:
    type: string
    description: Admin password
```

```
hidden: true
constraints:
  - length: { min: 6, max: 8 }
    description: Password length must be between 6 and 8 characters
  - allowed_pattern: "[a-zA-Z0-9]+"
    description: Password must consist of characters and numbers
      only
db_port:
  type: number
  description: Database port number
  default: 50000
  constraints:
    - range: { min: 40000, max: 60000 }
      description: Port number must be between 40000 and 60000

resources:
  server:
    type: OS::Nova::Server
    properties:
      key_name: { get_param: key_name }
      image:     { get_param: image }
      flavor:    { get_param: flavor }
      admin_pass: { get_param: admin_pass }
      user_data:
        str_replace:
          template: |
            #!/bin/bash
            echo db_port
        params:
          db_port: { get_param: db_port }

outputs:
  server_networks:
    description: The networks of the deployed server
    value: { get_attr: [server, networks] }
```

Environnement TME – DevStack

DevStack

Une VM contenant un Cloud OpenStack presque complet, installée et testée en salle 503.

Services installés : Keystone, Nova, Placement, Glance, Cinder, Neutron, Horizon.
Installation sur PC personnel :

- <https://www.devstack.org/>
- <https://docs.openstack.org/devstack/latest/>

Prérequis matériels :

- RAM : 4 Go minimum (8 Go recommandé, 32 Go en salle)
- Deux cartes réseaux :
 - Une en mode **NAT** ou **bridged** (accès Internet)
 - Une en mode **Host-only** avec l'adresse IP obligatoire 10.0.2.15/24
- L'adresse IP de l'hôte sur l'interface Host-only peut être 10.0.2.14/24

Sources

- OpenStack Documentation : <https://docs.openstack.org/>
- Project Navigator : <https://www.openstack.org/software/project-navigator/>
- DevStack : <https://docs.openstack.org/devstack/latest/>
- Flask-RESTful : <https://flask-restful.readthedocs.io/>
- Copyright Naceur.Malouch@lip6.fr – All rights reserved.